2026 年洛谷非专业级别收容能力认证

SCP-J/S 2026 第二轮

普及级++

时间：2026 年 10 月 18 日 08:00 ～ 12:00

题目名称	档案	染色	进位	数组
题目类型	传统型	传统型	传统型	传统型
目录	file	color	carry	array
可执行文件名	file	color	carry	array
输入文件名	file.in	color.in	carry.in	array.in
输出文件名	file.out	color.out	carry.out	array.out
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB
测试点数目	20	25	20	25
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	file.cpp	color.cpp	carry.cpp	array.cpp

编译选项

对于 C++ 语言	-O2 -std=c++14 -static

注意事项（请仔细阅读）

1. 文件名（程序名和输入输出文件名）必须使用英文小写。
2. C/C++ 中函数 main() 的返回值类型必须是 int，程序正常结束时的返回值必须是 0。
3. 提交的程序代码文件的放置位置请参考洛谷的具体要求。
4. 因违反以上三点而出现的错误或问题，申诉时一律不予受理。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目内存限制一致。
8. 只提供 Linux 格式附加样例文件。
9. 评测在当前最新公布的 NOI Linux 下进行，各语言的编译器版本以此为准。
10. **在洛谷提交时，无需使用文件读写。**

染色（color）

【题目描述】

小 W 有一个长度为 n 的正整数数组 A，从左至右的第 i 个数为 A_i。

A_i 将被染成红色或蓝色中的一种，不同的染色方案可能会导致不同的最终得分。设 $\{C_1, C_2, \cdots, C_n\}$ 为分数数组：

- 如果 A_i 左侧没有与其同色的数，则 $C_i = 0$。
- 否则，记其左侧**与其最靠近的同色数**为 A_j，则 $C_i = A_i - A_j$。

小 W 的染色得分为 $C_1 + C_2 + \cdots + C_n$，请你求出染色得分的最大值。

【输入格式】

从文件 ***color.in*** 中读入数据。

输入共两行。

第一行包含一个正整数 n，表示数组长度。

第二行包含 n 个正整数 $A_1, A_2, \ldots, A_n$。

【输出格式】

输出到文件 ***color.out*** 中。

输出一行包含一个整数，表示染色得分的最大值。

【样例 1 输入】

```
3
1 2 1
```

【样例 1 输出】

```
1
```

【样例 1 解释】

一个最优的染色方案为 1 2 1，此时最终得分为 1。可以证明没有更优的染色方案。

【样例 2 输入】

```
6
3 4 9 8 2 4
```

【样例 2 输出】

```
7
```

【样例 2 解释】

一个最优的染色方案为 3 4 9 8 2 4，此时最终得分为 7。可以证明没有更优的染色方案。

【样例 3】

见选手目录下的 ***color/color3.in*** 与 ***color/color3.ans***。
该样例满足测试点 16 ~ 20 的限制。

【样例 4】

见选手目录下的 ***color/color4.in*** 与 ***color/color4.ans***。
该样例满足测试点 21 ~ 25 的限制。

【数据范围】

对于所有测试点，保证：

- $2 \le n \le 2 \times 10^5$
- $1 \le A_i \le 3 \times 10^8$

测试点编号	$n \le$	特殊性质
$1 \sim 5$	3	无
$6 \sim 10$	15	
$11 \sim 15$	1000	
$16 \sim 20$	2×10^5	有
$21 \sim 25$		无

特殊性质：对于所有 $1 \le i \le n$，满足 $\frac{A_i}{i}$ 为定值。

进位（carry）

【题目描述】

给定一个长度为 n 的非负整数数组 $A_1, A_2, \ldots, A_n$。

对于区间 $l \sim r$，可以执行以下操作：

- 令 $S = 0$。
- 按照 $A_l, A_{l+1}, \ldots, A_r$ 的顺序，依次将 S 的值增加 A_i。

所有加法均按照普通的**二进制加法**进行，在一次二进制加法中，如果某一位向更高一位产生了一个进位，就称其发生了一次“进位”。不同二进制位产生的进位分别计算，同一次加法中发生的多次连续进位也分别计算。用 $C(l, r)$ 表示区间 $l \sim r$ 执行上述操作过程中发生的“进位”总数。

例如计算 $3 + 5$：$3 = 011_2$，$5 = 101_2$，相加得 1000_2。从低位到高位分别在第 $0, 1, 2$ 位产生进位，共 3 次进位。若发生连续进位，例如 $7 + 1 = 111_2 + 1_2 = 1000_2$，从第 0 位到第 2 位连续产生 3 次进位，也分别计入。

若一个长度为 m 的数组 $[p_1, p_2, \ldots, p_m]$ 满足以下条件，则称其合法：

- $m > 0$
- 对任意 $1 \le i \le m$，均满足 $1 \le p_i \le n$
- 对任意 $1 \le i < m$，均满足 $p_i < p_{i+1}$ 且 $C(p_i, p_{i+1}) = 1$

两个数组不同，当且仅当两个数组长度不同，或至少一个位置的数字的不同。

求有多少个不同的合法数组，答案对 $998,244,353$ 取模。

【输入格式】

从文件 ***carry.in*** 中读入数据。

第一行输入一个整数 T，表示测试数据组数。

对于每组测试数据，输入两行：

- 第一行输入一个整数 n。
- 第二行输入 n 个非负整数 $A_1, A_2, \ldots, A_n$。

【输出格式】

输出到文件 ***carry.out*** 中。

对于每组测试数据，输出一行一个非负整数，表示合法数组数量对 $998,244,353$ 取模后的结果。

【样例 1 输入】

```
3
5
1 4 4 3 8
6
6 3 7 4 6 6
5
52843137796 640771484120 983154867810 489455928042 397256148081
```

【样例 1 输出】

```
8
9
5
```

【样例 1 解释】

第一组数据合法下标序列为 [1], [2], [3], [4], [5], [1, 3], [2, 3], [2, 4]，共 8 个。
第二组数据合法下标序列为 [1], [2], [3], [4], [5], [6], [3, 4], [4, 5], [3, 4, 5]，共 9 个。
第三组数据合法下标序列为 [1], [2], [3], [4], [5]，共 5 个。

【样例 2】

见选手目录下的 ***carry/carry2.in*** 与 ***carry/carry2.ans***。
该样例满足测试点 3 ∼ 5 的限制。

【样例 3】

见选手目录下的 ***carry/carry3.in*** 与 ***carry/carry3.ans***。
该样例满足测试点 6 ∼ 8 的限制。

【样例 4】

见选手目录下的 ***carry/carry4.in*** 与 ***carry/carry4.ans***。
该样例满足测试点 9 ∼ 14 的限制。

【样例 5】

见选手目录下的 ***carry/carry5.in*** 与 ***carry/carry5.ans***。
该样例满足测试点 15 ∼ 16 的限制。

【样例 6】

见选手目录下的 ***carry/carry6.in*** 与 ***carry/carry6.ans***。
该样例满足测试点 $17 \sim 20$ 的限制。

【数据范围】

对于所有测试点，保证：

- $1 \le T \le 30$
- $1 \le n \le 5 \times 10^5$
- $0 \le A_i \le 10^{12}$
- $\sum n \le 6 \times 10^6$

其中，$\sum n$ 表示一个输入文件内所有测试数据中 n 的总和。

测试点编号	$n \le$	$\sum n \le$
$1, 2$	10	300
$3 \sim 5$	2×10^3	3×10^3
$6 \sim 8$	5×10^3	10^4
$9 \sim 14$	10^5	5×10^5
$15 \sim 16$	2×10^5	2×10^6
$17 \sim 20$	5×10^5	6×10^6

【提示】

在本题中，你可能需要快速求出一个整数在二进制表示下有多少位为 1。你可以使用如下内建函数：

- `__builtin_popcount(x)`：返回 x 的二进制表示中 1 的个数，参数 x 通常为 `int` 或 `unsigned int` 类型。
- `__builtin_popcountll(x)`：返回 x 的二进制表示中 1 的个数，参数 x 通常为 `long long` 或 `unsigned long long` 类型。

二者的时间复杂度均为 $O(1)$。

注意：本题输入量较大，建议选手使用较快的读入方式，以避免因输入耗时影响程序效率。

数组（array）

【题目描述】

若数组 a, S 长度均为 n，且对于所有 $1 \le i \le n$，均满足 $S_i = a_1 + a_2 + \cdots + a_i$，则数组 S 为数组 a 的前缀和数组。

小 D 有 n 个数组 $A_0, A_1, \cdots, A_{n-1}$，长度均为 n。用 $A_{i,j}$ 表示数组 A_i 的第 j 项。已知数列 A_0 的初始值，对于所有 $1 \le i < n$，数组 A_i 为数组 A_{i-1} 的前缀和数组。

现在小 D 有 p 个修改操作和 q 个查询操作：

- 对于修改操作，给出 x, l, r, k。将数组 A_x 的第 $l \sim r$ 项均增加 k。增加后，重新计算除了数组 A_x 外的所有数组，使得对所有 $1 \le i < n$，满足数组 A_i 为数组 A_{i-1} 的前缀和数组。
- 对于查询操作，给出 x, l, r。请求出数组 A_x 中第 $l \sim r$ 项之和。结果对 $10^9 + 7$ 取模。

请你执行这些操作，并输出所有查询操作的结果。

【输入格式】

从文件 ***array.in*** 中读入数据。

输入的第一行包含三个整数 n, p, q，分别表示数组数量，修改操作数量和询问操作数量。

输入的第二行包含 n 个整数 $A_{0,1}, A_{0,2}, \ldots, A_{0,n}$，表示初始的数列 A_0。

接下来 $p + q$ 行，每行输入第一个数字 o 表示操作种类：

- 若 $o = 1$，则为修改操作。接下来输入四个数字 x, l, r, k，表示将数组 A_x 的第 $l \sim r$ 项增加 k。
- 若 $o = 2$，则为查询操作。接下来输入三个数字 x, l, r，表示查询数组 A_x 中第 $l \sim r$ 项之和。

【输出格式】

输出到文件 ***array.out*** 中。

输出 q 行，一行一个非负整数，表示查询操作的结果。结果对 $10^9 + 7$ 取模。

【样例 1 输入】

```
10 2 3
1 2 3 4 5 6 7 8 9 10
1 0 2 4 1
2 0 2 4
```

```
2 1 2 4
1 1 3 5 -1
2 2 2 8
```

【样例 1 输出】

```
12
25
378
```

【样例 1 解释】

第一次修改后：

- 原数组 A_0 为 $[1,3,4,5,5,6,7,8,9,10]$；
- A_1 为 $[1,4,8,13,18,24,31,39,48,58]$；
- 查询的答案分别为 $3+4+5=12$ 和 $4+8+13=25$。

第二次修改后：

- A_1 为 $[1,4,7,12,17,24,31,39,48,58]$；
- A_2 为 $[1,5,12,24,41,65,96,135,183,241]$；
- 第 2 项到第 8 项的和为 $5+12+24+41+65+96+135=378$。

【样例 2】

见选手目录下的 ***array/array2.in*** 与 ***array/array2.ans***。
该样例满足测试点 $4 \sim 7$ 的限制。

【样例 3】

见选手目录下的 ***array/array3.in*** 与 ***array/array3.ans***。
该样例满足测试点 $8 \sim 10$ 的限制。

【样例 4】

见选手目录下的 ***array/array4.in*** 与 ***array/array4.ans***。
该样例满足测试点 $22 \sim 25$ 的限制。

【数据范围】

对于所有测试点，保证：

- $1 \le n \le 300$
- $0 \le p, q \le 10^5$
- 对于所有 $1 \le i \le n$，均有 $|A_{0,i}| \le 10^9$
- $o \in \{1, 2\}$
- 对于所有操作，均有 $0 \le x < n$，$1 \le l \le r \le n$
- $|k| \le 10^9$

测试点编号	$n \le$	特殊性质
$1 \sim 3$	10	$p, q \le 10$
$4 \sim 7$	50	无
$8 \sim 10$	300	$x = 0$
$11, 12$		$p = 0$
$13, 14$		$q = 1$
$15, 16$		询问中的 $x = 0$
$17, 18$		修改中的 $x = 0$
$19 \sim 21$		$p, q \le 3000$
$22 \sim 25$		无

档案（file）

【题目描述】

小 Z 正在整理一批档案。

初始时，有 n 个**核心档案**已经被归档，我们用长度为 n 的序列 a 表示这 n 个核心档案的编号。此外，还有 m 批待归档的档案，其中第 i 批档案的编号用**序列** b_i 表示。

已知所有档案的编号**互不相同**，即序列 $a, b_1, b_2, \ldots, b_{m-1}, b_m$ 中包含的所有元素两两不同。

初始时 n 个**核心档案**已经被归档，其余档案均未被归档。记当前的**已归档档案序列**为 x，初始时 $x = a$。

小 Z 需要执行 m 次归档操作，每次操作依次进行以下步骤：

1. 选择尚未归档的一批档案 b_i，将其中的所有元素加入序列 x。
2. 将序列 x 中的元素从小到大排序。

由于核心档案比较重要，小 Z 希望他们在所有档案中尽可能靠前以方便调阅。因此小 Z 规定本次操作的代价为排序后序列 x 中所有核心档案的下标之和。所有序列的下标从 1 开始编号。

进行全部 m 次操作的总代价为每次操作的代价之和。

小 Z 可以任意决定这 m 批档案的处理顺序，请你求出执行全部 m 次操作的最小总代价。

【输入格式】

从文件 ***file.in*** 中读入数据。

第一行输入两个正整数 n, m，表示核心档案的数量与待归档档案的批次数。

第二行输入 n 个正整数，表示核心档案序列 a。

接下来 m 行，每行描述一批待归档档案：

- 首先输入一个正整数 l_i 表示档案序列 b_i 的长度。
- 随后输入 l_i 个正整数表示档案序列 b_i。

【输出格式】

输出到文件 ***file.out*** 中。

输出一行一个正整数，表示执行全部 m 次操作的最小总代价。

【样例 1 输入】

```
2 2
2 5
```

```
2 1 3
2 4 6
```

【样例 1 输出】

```
11
```

【样例 1 解释】

- 我们可以先对档案序列 $[4,6]$ 进行归档，得到已归档档案序列为 $[2,4,5,6]$，本次操作的代价为 $1+3=4$。
- 接着对档案序列 $[1,3]$ 进行归档，得到已归档档案序列为 $[1,2,3,4,5,6]$，本次操作的代价为 $2+5=7$。

总代价为 $4+7=11$。

- 我们也可以先对档案序列 $[1,3]$ 进行归档，得到已归档档案序列为 $[1,2,3,5]$，本次操作的代价为 $2+4=6$。
- 接着对档案序列 $[4,6]$ 进行归档，得到已归档档案序列为 $[1,2,3,4,5,6]$，本次操作的代价为 $2+5=7$。

总代价为 $6+7=13$。

因此最小总代价为 11。

【样例 2 输入】

```
5 4
7 10 15 19 25
1 1
3 16 21 20
3 12 17 29
6 5 9 11 18 24 26
```

【样例 2 输出】

```
116
```

【样例 2 解释】

可以依次归档第 $2,1,3,4$ 批档案，四次操作的代价分别为 $19,24,29,44$，总代价为 $19+24+29+44=116$。可以证明，不存在总代价更小的归档顺序。

【样例 3】

见选手目录下的 ***file/file3.in*** 与 ***file/file3.ans***。
该样例满足测试点 $1 \sim 2$ 的条件限制。

【样例 4】

见选手目录下的 ***file/file4.in*** 与 ***file/file4.ans***。
该样例满足测试点 $3 \sim 5$ 的条件限制。

【样例 5】

见选手目录下的 ***file/file5.in*** 与 ***file/file5.ans***。
该样例满足测试点 $6 \sim 8$ 的条件限制。

【样例 6】

见选手目录下的 ***file/file6.in*** 与 ***file/file6.ans***。
该样例满足测试点 $9 \sim 12$ 的条件限制。

【样例 7】

见选手目录下的 ***file/file7.in*** 与 ***file/file7.ans***。
该样例满足测试点 13 的条件限制。

【样例 8】

见选手目录下的 ***file/file8.in*** 与 ***file/file8.ans***。
该样例满足测试点 14 的条件限制。

【样例 9】

见选手目录下的 ***file/file9.in*** 与 ***file/file9.ans***。
该样例满足测试点 $15 \sim 20$ 的条件限制。

【数据范围】

设 $L = \sum_{i=1}^{m} l_i$，表示所有待归档档案的数量。记 $b_{i,j}$ 为序列 b_i 的第 j 项。对于所有测试数据，保证：

- $1 \le n, m, L \le 10^6$；
- 对于所有 $1 \le i \le n$，均有 $1 \le a_i \le 2 \times 10^6$；

- 对于所有 $1 \le i \le m$，均有 $1 \le l_i \le 10^6$；
- 对于所有 $1 \le i \le m$，$1 \le j \le l_i$，均有 $1 \le b_{i,j} \le 2 \times 10^6$；
- 序列 $a, b_1, b_2, \ldots, b_{m-1}, b_m$ 中包含的所有元素两两不同。

测试点编号	$m \le$	$n, L \le$	特殊性质
$1 \sim 2$	7	100	无
$3 \sim 5$	11	1 000	
$6 \sim 8$	100	10^4	
$9 \sim 12$	10^5	10^5	
13	10^6	10^6	A
14			B
$15 \sim 20$			无

特殊性质 A：$n = 1$。

特殊性质 B：

- 对于所有 $1 \le i \le m$，$1 \le j < l_i$，有 $b_{i,j} + 1 = b_{i,j+1}$。即每批待归档档案的编号是连续的。
- 对于所有 $1 \le i < n$，有 $a_i + 1 = a_{i+1}$。即核心档案的编号也是连续的。

函数（func）

【题目描述】

给定一个长度为 n 的非负整数序列 a。

对于非空的非负整数集合 $S = \{s_1, s_2, \cdots, s_k\}$，如下定义两个函数：

- 定义 $\mathrm{OR}(S) = s_1 \vee s_2 \vee \cdots \vee s_k$，即集合 S 中所有元素进行按位或运算所得的结果。其中 $\vee$ 表示按位或运算。
- 定义 $\mathrm{MEX}(S) = \min\{x \in \mathbb{N} \mid x \notin S\}$，即不在集合 S 中的最小非负整数。

对于区间 $[l, r]$（$1 \le l \le r \le n$），如下定义它的价值 $f(l, r)$：

- 设 $S = \{a_l, a_{l+1}, \cdots, a_r\}$；
- $f(l, r) = \mathrm{OR}(S) \vee \mathrm{MEX}(S)$。其中 $\vee$ 表示按位或运算。

你的任务是求出 $\sum_{1 \le l \le r \le n} f(l, r)$，即所有区间的价值和。答案对 2^{64} 取模。

【输入格式】

从文件 ***func.in*** 中读入数据。

本题输入量较大，请注意选择较快的输入方式。

输入的第一行包含一个正整数 n，表示序列的长度。

输入的第二行包含 n 个非负整数 $a_1, a_2, \ldots, a_n$。

【输出格式】

输出到文件 ***func.out*** 中。

输出一行一个非负整数，表示所有区间的价值之和对 2^{64} 取模后的结果。

【样例 1 输入】

```
3
0 1 2
```

【样例 1 输出】

```
13
```

【样例 1 解释】

序列为 $[0, 1, 2]$，共有 6 个区间。

以区间 $[1,2]$ 为例，其中的数构成集合 $S=\{0,1\}$，有 $\mathrm{OR}(S)=1$，$\mathrm{MEX}(S)=2$，因此 $f(1,2)=1\vee 2=3$。

各个区间的价值分别为：

- $f(1,1)=1$，$f(2,2)=1$，$f(3,3)=2$；
- $f(1,2)=3$，$f(2,3)=3$；
- $f(1,3)=3$。

因此，所有区间的价值之和为 $1+1+2+3+3+3=13$。

【样例 2 输入】

```
4
1 0 1 3
```

【样例 2 输出】

```
24
```

【样例 2 解释】

序列为 $[1,0,1,3]$，共有 10 个区间。

以区间 $[1,3]$ 为例，其中的数为 $1,0,1$，构成的集合为 $S=\{0,1\}$。有 $\mathrm{OR}(S)=1$，$\mathrm{MEX}(S)=2$，因此 $f(1,3)=1\vee 2=3$。

长度为 1 的四个区间的价值依次为 $1,1,1,3$，其余六个区间的价值均为 3。

因此，所有区间的价值之和为 $1+1+1+3+6\times 3=24$。

【样例 3 输入】

```
10
536870912 0 1 2 3 536870912 1073741823 0 0 1
```

【样例 3 输出】

```
35970351152
```

【样例 4】

见选手目录下的 ***func/func4.in*** 与 ***func/func4.ans***。

该样例满足测试点 3 的条件限制。

【样例 5】

见选手目录下的 ***func/func5.in*** 与 ***func/func5.ans***。
该样例满足测试点 8 的条件限制。

【样例 6】

见选手目录下的 ***func/func6.in*** 与 ***func/func6.ans***。
该样例满足测试点 12 的条件限制。

【样例 7】

见选手目录下的 ***func/func7.in*** 与 ***func/func7.ans***。
该样例满足测试点 17 ∼ 20 的条件限制。

【数据范围】

对于所有测试数据，保证：

- $1 \le n \le 2 \times 10^6$；
- 对于所有 $1 \le i \le n$，均有 $0 \le a_i < 2^{30}$。

<table>
<tr><th>测试点编号</th><th>$n \le$</th><th>$a_i <$</th><th>特殊性质</th></tr>
<tr><td>1</td><td>10</td><td>4</td><td rowspan="4">×</td></tr>
<tr><td>2</td><td>100</td><td>2^{10}</td></tr>
<tr><td>3</td><td>1 000</td><td rowspan="2">2^{20}</td></tr>
<tr><td>4</td><td>3 000</td></tr>
<tr><td>5 ∼ 6</td><td>5 000</td><td rowspan="3">2^{30}</td><td rowspan="2">✓</td></tr>
<tr><td>7</td><td>2×10^4</td></tr>
<tr><td>8</td><td>5×10^4</td><td rowspan="5">×</td></tr>
<tr><td>9 ∼ 10</td><td rowspan="2">10^5</td></tr>
<tr><td>11</td><td>2^{10}</td></tr>
<tr><td>12</td><td rowspan="2">2×10^5</td><td>2^{20}</td></tr>
<tr><td>13</td><td rowspan="2">2^{30}</td></tr>
<tr><td>14</td><td>5×10^5</td></tr>
<tr><td>15</td><td>10^6</td><td>2^{10}</td><td rowspan="2">✓</td></tr>
<tr><td>16</td><td>1.5×10^6</td><td>2^{20}</td></tr>
<tr><td>17 ∼ 20</td><td>2×10^6</td><td>2^{30}</td><td>×</td></tr>
</table>

特殊性质：对于所有 $1 \le i \le n$，a_i 在对应测试点规定的取值范围内独立、均匀随机生成。